NATIONAL ECONOMICS UNIVERSITY, HANOI
COLLEGE OF TECHNOLOGY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE



INTRODUCTION TO AI (TOKT11121.AI67A)

REPORT

Topic: Smart shopping

Instructor: PhD. Nguyen Trong Nghia

Student:	Le Hai Yen	11259127
	Phan Thanh Thao	11257758
	Nguyen Thi Thao Uyen	11258816
	Tong Thoai Khanh	11253781
	Pham Le Bao Quynh	11257321

Hanoi, November 2026

TABLE OF CONTENTS

1. Introduction **3**
2. Problem Formulation **4**
2.1. State Representation 4
2.2. Formulas Used in the Report 4
3. Algorithm **5**
3.1. Market/Supermarket List Generation 5
3.2. Search Method 9
3.2.1. Greedy Algorithm (Traveling to the nearest market/supermarket) 9
3.2.2. A* Algorithm (A* Search) 10
a. How the Heuristic value is calculated 10
b. A specific example of how to calculate heuristics 10
c. A* Algorithm 13
3.3 Performance Evaluation and Measurement Criteria 14
4. Experiments and Results **14**
4.1. Test Scenario Setup 14
4.2. Simulation Runs and Data Collection 15
4.3. Performance Analysis 17
5. Discussion **18**
5.1. Overall Trade-offs Evaluation 18
5.2. Real-world Applications and Implementation Scenarios 20
5.3. Case Study: Smart Grocery Routing at Grab 20
6. Conclusion **21**
6.1. Summary of the Study 21
a. Pros and Cons of each algorithm: 21
b. Conclusion on the Practicality of Greedy (Nearest Neighbor): 22
6.2. Future Works 22
6.3. Final Remarks 23

1. Introduction

During every gathering or preparation for a traditional feast, such as the Tet holiday or festivals, families often face a headache because the ingredient list is long and items are sold in scattered locations. One place sells vegetables, another sells meat, and supermarkets have entirely different items. This often leads to driving around in circles, wasting gas and time. Therefore, our team came up with the idea of creating a "Smart Grocery Routing" problem to use AI to find a route that ensures all necessary ingredients for the feast are bought while traveling the shortest possible distance.

Problem Modeling: We modeled this problem as finding a path through various points. In this model:

- The starting point is Home at coordinates $(0, 0)$.
- The destinations are markets/supermarkets around the area.
- Each market/supermarket has coordinates (x, y) measured in km.
- The distance between points is calculated based on the coordinate system (Euclidean distance).
- When visiting a location, the buyer purchases all missing ingredients that are sold there.
- **Requirement:** Find the shortest total distance to buy all necessary ingredients (starting from Home, passing through the necessary locations, and returning Home).

Algorithms Used: To solve this problem, our team implemented and compared two classic search algorithms:

- **Greedy Algorithm (Nearest Neighbor):** From the current location, always go to the nearest market (which mirrors human instinct).
- **A* Algorithm:** Slightly smarter, as it simultaneously calculates the distance already traveled and uses a heuristic function to estimate the remaining distance, finding the globally optimal path rather than just the immediate best choice.

In summary, the goals of this report are:

- To model daily grocery shopping by coordinate-mapping the necessary market locations.
- To apply and compare the Greedy and A* algorithms across multiple maps of varying scales.
- To evaluate algorithm performance based on criteria concerning efficiency, optimality, and resource consumption.
- To draw insights that can guide the grocery planning of homemakers in reality and optimize travel costs based on artificial intelligence (AI).

Through this analysis, we aim to bridge the gap between theory and practice, further applying existing algorithms to optimize delivery routing, warehouse placement, or the daily grocery shopping of homemakers.

2. Problem Formulation

To simulate real-world routing scenarios, we coordinate-mapped the market/supermarket locations on a map. We used a 2-dimensional space, and during our survey, all connecting routes from the markets/supermarkets allow for direct travel back Home without needing to pass through other locations.

*** Assume the path between the two locations is a straight line and the route is unobstructed.**

2.1. State Representation

- **Home:** Located at $(0, 0)$, representing the starting point, traveling through the required stops, and returning to Home.
- The destination markets and supermarkets are represented as coordinates (x, y).
- Once a point has been visited, it cannot be revisited.

2.2. Formulas Used in the Report

The distance between 2 nodes is calculated based on the "Euclidean Distance" formula:

$$d(A, B) = \sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$$

Heuristics Calculation Formula:

- **Case 1:** If $S = \emptyset$

$$h(p, S) = d(p, N)$$

- **Case 2:** If $S \neq \emptyset$

$$h(p, S) = \max_{i \in S} \left(\min_{m \in sells(i)} [d(p, m) + d(m, N)] \right)$$

Where:

- **p** : Current location
- **N** : Starting point (Home)
- **S** : Set of missing ingredients
- ***d(a,b)*** : Euclidean distance between 2 points
- **i** : An ingredient needed in S
- **m** : Market/supermarket points

- ***sells(i)*** : Set of markets/supermarkets that have ingredient *i*
- ***h(p, S)*** : Heuristic value of node *p* given the set of missing ingredients *S*

3. Algorithm

3.1. Market/Supermarket List Generation

All inputs were created in a format consisting of one line for the required ingredients and one table with 3 columns: Market/supermarket name, coordinates, and available ingredients. A total of 5 inputs were used in the report, with a progressively increasing number of locations or an increasing number of markets carrying similar ingredients. Below are the 5 inputs used to test the algorithms:

Input 1:

Ingredients to buy: Thịt heo, Cá, Rau muống, Cà chua, Trứng, Hành lá, Gạo, Nước mắm

Market / Supermarket	Coordinates	Available Ingredients
Chợ Hoa Phượng	(2, 1)	Rau muống, Cà chua, Hành lá
Chợ Đầu Mối	(5, 2)	Thịt heo, Cá, Trứng
Siêu thị Xanh	(6, 5)	Gạo, Nước mắm, Trứng, Hành lá
Chợ Cầu	(8, 4)	Cá, Rau muống, Cà chua
Siêu thị Mini 24h	(1, 6)	Nước mắm, Trứng

Input 2:

Ingredients to buy: Thịt bò, Thịt gà, Tôm, Cá, Rau cải, Cà rốt, Khoai tây, Hành tây, Trứng, Sữa, Dầu ăn, Nước mắm

Market / Supermarket	Coordinates	Available Ingredients
Chợ Sớm	(1, 2)	Rau cải, Cà rốt
Chợ Hải Sản	(6, 1)	Tôm, Cá

Chợ Trung Tâm	(5, 5)	Thịt bò, Thịt gà, Hành tây, Khoai tây, Rau cải
Siêu thị BigMart	(9, 7)	Thịt bò, Thịt gà, Tôm, Cá, Cà rốt, Khoai tây, Hành tây, Trứng, Sữa, Dầu ăn, Nước mắm
Cửa hàng Tiện Lợi	(2, 5)	Sữa, Dầu ăn, Trứng
Chợ Đông	(8, 3)	Cá, Khoai tây, Nước mắm, Dầu ăn
Chợ Nhỏ Góc Phố	(3, 1)	Hành tây, Cà rốt

Input 3:

Ingredients to buy: Thịt heo, Thịt bò, Thịt gà, Cá, Tôm, Mực, Rau muống, Cà chua, Bắp cải, Hành tây, Trứng, Gạo, Nước mắm, Dầu ăn, Sữa

Market / Supermarket	**Coordinates**	**Available Ingredients**
Chợ Sáng	(1, 3)	Rau muống, Cà chua, Bắp cải
Chợ Cá Bến	(7, 1)	Cá, Tôm, Mực
Chợ Thịt Phố	(4, 6)	Thịt heo, Thịt bò, Thịt gà
Siêu thị Mega	(12, 9)	Thịt heo, Thịt bò, Thịt gà, Cá, Tôm, Rau muống, Bắp cải, Hành tây, Trứng, Gạo, Nước mắm, Dầu ăn, Sữa
Cửa hàng Tạp Hóa	(2, 1)	Gạo, Nước mắm, Dầu ăn
Chợ Đồi	(9, 6)	Cà chua, Hành tây, Trứng, Thịt gà
Siêu thị Mini	(3, 8)	Sữa, Trứng, Dầu ăn
Chợ Vườn	(6, 3)	Rau muống, Cà chua, Hành tây, Bắp cải

Chợ Hải Sản Đông	(11, 3)	Tôm, Mực, Cá, Nước mắm
Siêu thị Sữa & Trứng	(8, 9)	Sữa, Trứng, Gạo

Input 4:

Ingredients to buy: Thịt heo, Thịt bò, Thịt gà, Cá, Tôm, Rau muống, Cà chua, Hành tây, Bắp cải, Cà rốt, Khoai tây, Trứng, Gạo, Nước mắm, Dầu ăn, Đường, Muối, Sữa

Market / Supermarket	**Coordinates**	**Available Ingredients**
Chợ Rau Xanh	(4, 5)	Rau muống, Cà chua, Bắp cải, Cà rốt
Chợ Thịt Phố	(4, 4)	Thịt heo, Thịt bò, Thịt gà
Chợ Hải Sản	(5, 2)	Cá, Tôm, Nước mắm
Tạp Hóa Cô Lan	(9, 7)	Gạo, Dầu ăn, Đường, Muối, Nước mắm
Siêu thị Sữa & Trứng	(0, 5)	Sữa, Trứng, Đường
Chợ Tổng Hợp	(6, 4)	Thịt heo, Thịt gà, Cá, Rau muống, Cà chua, Hành tây, Khoai tây, Trứng
Siêu thị Tiện Ích	(2, 1)	Gạo, Dầu ăn, Muối, Sữa, Trứng, Khoai tây, Hành tây
Siêu thị Mega	(8, 12)	Thịt heo, Thịt bò, Thịt gà, Cá, Rau muống, Cà chua, Hành tây, Bắp cải, Khoai tây, Trứng, Gạo, Nước mắm, Dầu ăn, Sữa, Muối
Chợ Nhỏ Góc Phố	(10, 8)	Cà rốt, Hành tây
Cửa hàng 24h	(5, 10)	Sữa, Trứng, Muối
Chợ Cầu Vồng	(8, 9)	Cá, Gạo, Khoai tây
Chợ Làng	(9, 10)	Rau muống, Cà chua, Cà rốt, Hành tây

Input 5:

Ingredients to buy: Thịt heo, Thịt bò, Thịt gà, Cá, Tôm, Mực, Rau muống, Cải ngọt, Cà chua, Hành tây, Bắp cải, Cà rốt, Khoai tây, Nấm, Đậu phụ, Trứng, Gạo, Bún, Nước mắm, Dầu ăn, Đường, Muối, Sữa, Bột mì

Market / Supermarket	Coordinates	Available Ingredients
Chợ Làng	(6, 12)	Rau muống, Cải ngọt, Cà chua, Cà rốt
Chợ Thịt Phố	(8, 1)	Thịt heo, Thịt bò, Thịt gà, Hành tây
Chợ Cá Bến	(6, 1)	Cá, Tôm, Mực, Nước mắm
Tạp Hóa Cô Lan	(10, 15)	Gạo, Bún, Nước mắm, Dầu ăn, Đường, Muối
Chợ Xanh	(5, 3)	Rau muống, Cải ngọt, Bắp cải, Cà rốt, Hành tây, Nấm
Siêu thị An Bình	(10, 0)	Trứng, Sữa, Gạo, Dầu ăn, Bột mì, Đường
Chợ Đậu & Nấm	(4, 4)	Đậu phụ, Nấm, Bún, Cải ngọt
Siêu thị Hoàng Gia	(6, 5)	Thịt heo, Thịt gà, Cá, Tôm, Rau muống, Cải ngọt, Cà chua, Hành tây, Bắp cải, Khoai tây, Trứng, Gạo, Dầu ăn, Sữa, Đường, Muối, Bột mì, Bún
Siêu thị Mega	(15, 2)	Thịt bò, Mực, Nấm, Đậu phụ, Nước mắm, Thịt heo, Trứng, Gạo, Dầu ăn, Sữa, Khoai tây, Cà rốt
Chợ Đêm	(5, 6)	Trứng, Sữa, Thịt gà, Khoai tây
Chợ Đồi	(15, 9)	Cà chua, Hành tây, Trứng, Thịt gà, Khoai tây
Cửa hàng 24h	(10, 9)	Sữa, Trứng, Dầu ăn, Muối

Chợ Hải Sản Đông	(14, 1)	Tôm, Mực, Cá, Nước mắm
Siêu thị Mini	(12, 13)	Bột mì, Đường, Sữa, Trứng
Chợ Cầu	(5, 1)	Khoai tây, Cà rốt, Bắp cải, Đậu phụ

3.2. Search Method

To solve the shortest path problem across the markets, two representative search algorithms were selected and implemented in this study: the Greedy algorithm (Nearest Neighbor) and the A* algorithm (A* Search).

3.2.1. Greedy Algorithm (Traveling to the nearest market/supermarket)

```python
def greedy(home, markets, need):
    check_feasible(markets, need)
    pos, left = home, set(need)
    route, total = [], 0.0
    while left:
        # Consider only those locations that still sell the missing ingredients
        # In the event of a tie in distance, choose the location that appears first
        d, i = min(
            (dist(pos, m["pos"]), i)
            for i, m in enumerate(markets)
            if left & m["items"]
        )
        total += d
        pos = markets[i]["pos"]
        route.append(markets[i]["name"])
        left -= markets[i]["items"]  # Buy all available items at this market
    total += dist(pos, home)
    return route, total
```

The Greedy (Nearest Neighbor) algorithm is used to find a shopping route across the markets such that the total travel distance is minimized. The algorithm starts from the Home location, then continuously selects the nearest market to the current position among those that still provide at least one missing ingredient. After arriving at a market, the algorithm updates the current position, records the market in the route, and removes the purchased ingredients from the needed list. This process continues until all necessary ingredients are fully purchased. Finally, the algorithm calculates the additional distance from the last market back home to determine the total route distance.

3.2.2. A* Algorithm (A* Search)

a. How the Heuristic value is calculated

```
def heuristic(pos, left, home, markets):
    if not left:
        return dist(pos, home)
    return max(
        min(
            dist(pos, m["pos"]) + dist(m["pos"], home)
            for m in markets
            if item in m["items"]
        )
        for item in left
    )
```

The Heuristic function estimates the distance based on the current location, the list of unpurchased ingredients, the home location, and the list of markets.

Specifically, the algorithm examines each missing ingredient, finds the nearest market selling that ingredient, and calculates the total distance from the current position to that market, and then from that market back home. Finally, the algorithm takes the **maximum value** among these estimated distances as the Heuristic value.

If there are no remaining ingredients to buy, the algorithm only calculates the distance from the current position back home.

This heuristic calculation method ensures that all inputs are admissible; thus, all routes found via the A* algorithm are inherently optimal.

b. A specific example of how to calculate heuristics

Heuristic formula:

- **Case 1:** If $S = \emptyset \Rightarrow h(p, S) = d(p, N)$
- **Case 2:** If $S \neq \emptyset \Rightarrow h(p, S) = max_{i \in S}\left(min_{m \in sells(i)}[d(p, m) + d(m, N)]\right)$

  Where:

  - **p** : Current location
  - **N** : Starting point (Home)
  - **S** : Set of missing ingredients
  - ***d(a,b)*** : Euclidean distance between 2 points
  - **i** : An ingredient needed in S
  - **m** : Market/supermarket points
  - ***sells(i)*** : Set of markets/supermarkets that have ingredient i

- ***h(p, S)*** : Heuristic value of node p given the set of missing ingredients S

Meaning of the formula: For each missing ingredient, choose the selling location that results in the shortest trip: $p \rightarrow$ location $\rightarrow$ Home. Then take the max across the ingredients (without accumulating to avoid double-counting trips).

Taking Input 1 as a specific example for calculation:

Calculating the Heuristic value of Home $(0, 0)$: When at Home, lacking all 8 ingredients:

Ingredient	**Selling Locations (total d(p, m) + d(m, Home))**	**c(i) = min**
Thịt heo	Đầu Mối 10,77	10,77
Cá	Đầu Mối 10,77; Chợ Cầu 17,89	10,77
Rau muống	Hoa Phượng 4,47; Chợ Cầu 17,89	4,47
Cà chua	Hoa Phượng 4,47; Chợ Cầu 17,89	4,47
Hành lá	Hoa Phượng 4,47; Siêu thị Xanh 15,62	4,47
Trứng	Đầu Mối 10,77; Mini 24h 12,17; Xanh 15,62	10,77
Nước mắm	Mini 24h 12,17; Xanh 15,62	12,17
Gạo	Siêu thị Xanh 15,62 (chỉ có nơi này)	15,62

Comparing the last column, the maximum is 15.62 (Gạo), so $h = 15.62$.

How to choose the next destination:

Continuing using input 1, after calculate h for Home, we consider all potential next markets/supermarkets to visit, based on the h of each market/supermarket and the ingredients that would remain missing after completing purchases there

For example, calculating h for Chợ Hoa Phượng: Considering Chợ Hoa Phượng, 3 ingredients have been acquired, while 5 are still missing:

Missing Item	**Selling Locations: d(Hoa Phượng, m) + d(m, Home)**	**min**
Thịt heo	Chợ Đầu Mối: 3,16 + 5,39 = 8,55	8,55

Cá	Chợ Đầu Mối: 8,55; Chợ Cầu: 6,71 + 8,94 = 15,65	8,55
Trứng	Chợ Đầu Mối: 8,55; Siêu thị Xanh : 5,66 + 7,81 = 13,47; Siêu thị Mini 24h : 5,10 + 6,08 = 11,18	8,55
Nước mắm	Siêu thị Xanh: 13,47; Siêu thị Mini 24h : 11,18	11,18
Gạo	Siêu thị Xanh: 5,66 + 7,81 = 13,47	13,47

Taking the max, we get $h = 13.47$. Doing the same, we have the h table for the potential next markets/supermarkets:

Destination	**g**	**h (after arrival)**	**f = g + h**
Chợ Hoa Phượng	2,24	13,47	15,70
Chợ Đầu Mối	5,39	10,97	16,36
Siêu thị Xanh	7,81	8,55	16,36
Chợ Cầu	8,94	10,05	18,99
Siêu thị Mini 24h	6,08	12,91	18,99

A* chooses Chợ Hoa Phượng (smallest f), rather than going straight to Siêu thị Xanh even though Gạo was the deciding ingredient for the initial h. From there, we repeat the process for the remaining steps until all ingredients are bought.

c. A* Algorithm

```
def astar(home, markets, need):
    check_feasible(markets, need)
    need = frozenset(need)
    tie = itertools.count()
    def position(loc):
        return home if loc == HOME else markets[loc]["pos"]

    start = (HOME, need)
    open_heap = [(heuristic(home, need, home, markets), next(tie), 0.0, HOME, need, ())]
    best_g = {}
    expanded = 0

    while open_heap:
        f, _, g, loc, left, path = heapq.heappop(open_heap)

        # Skip if this state has already been encountered with a g-value that is better than or equal to the current one
        if best_g.get((loc, left), math.inf) <= g:
            continue
        best_g[(loc, left)] = g
        expanded += 1

        # Check the goal: all items are purchased and brought home
        if not left and loc == HOME and path:
            return [markets[i]["name"] for i in path], g, expanded

        pos = position(loc)

        if not left:
            ng = g + dist(pos, home)
            heapq.heappush(open_heap, (ng, next(tie), ng, HOME, left, path))
            continue

        # In the event that materials are missing, head to a location that sells at least one of them
        for i, m in enumerate(markets):
            if i == loc or not (left & m["items"]):
                continue
            new_left = left - m["items"]
            ng = g + dist(pos, m["pos"])
            nf = ng + heuristic(m["pos"], new_left, home, markets)
            heapq.heappush(open_heap, (nf, next(tie), ng, i, new_left, path + (i,)))

    return None, math.inf, expanded  # No solution found
```

The A* algorithm is used to find a shopping route with an optimal total distance by **combining the actual travel cost and the estimated remaining cost**. The algorithm begins at home, identifies the list of ingredients to buy, and pushes the initial state into a priority queue. At each step, the algorithm selects the state with the lowest estimated total cost, calculated by the formula

$$f(n) = g(n) + h(n)$$

From the current location, the algorithm considers markets selling at least one missing ingredient, calculates the travel distance, and updates the list of needed ingredients. New states are pushed into the priority queue for further evaluation and selection. Once all ingredients have been bought, the algorithm calculates the extra distance back home. The search concludes when it finds a state satisfying the condition that all ingredients are bought and it has returned to the starting location. The returned output includes the list of visited markets, the total distance traveled, and the number of expanded states.

3.3. Visualization:

3.4. Performance Evaluation and Measurement Criteria

To ensure performance evaluation was fair and consistent, we built a benchmarking function to execute each algorithm across multiple randomly generated maps. The following evaluation criteria were recorded:

- **Average Execution Time (seconds)** — A metric for computational efficiency.
- **Peak Memory Usage (bytes)** — An indicator of the spatial complexity of each algorithm.
- **Path Optimality (number of steps)** — A measure of how close the found route is to the actual shortest path.

This multi-criteria analysis method allows for both quantitative and qualitative comparisons, thereby highlighting the trade-offs between computational cost, accuracy, and responsiveness.

4. Experiments and Results

4.1. Test Scenario Setup

The team conducted experiments on 5 datasets (inputs) with a gradually increasing number of locations and ingredients, to assess the performance and scalability of the Greedy (Nearest Neighbor) and A* algorithms in the shortest shopping routing problem. The datasets encompass 5, 7, 10, 12, and 15 locations, corresponding to 8, 12, 15, 18, and 24 types of required ingredients. For each dataset:

- Both the Greedy and A* algorithms were run on the same dataset to ensure fair comparison.
- Each test was performed 20 times, taking the average execution time.
- **Peak Memory Usage** was recorded to evaluate the resources consumed by each algorithm.
- The number of movement steps, total distance, and path optimality degree were calculated and compared.
- For the A* algorithm, the number of **Expanded Nodes** was also recorded to evaluate the search process.

All algorithms were implemented in Python using the same input data system. Conducting multiple experimental runs helps mitigate the effects of measurement variance, thereby providing a basis for comparing processing speed, memory consumption, and the ability to find optimal routes between the two algorithms as problem scale increases.

4.2. Simulation Runs and Data Collection

a. 5 locations, 8 ingredients to buy:

```
=== input1: 5 places, 8 ingredients (time = average of 20 runs) ===
Criterion                                   Greedy          A*
------------------------------------------------------------------
Average execution time (s)                0.000013    0.000130
Peak memory usage (bytes)                    1,664       7,808
Steps (moves, incl. return home)                 4           4
Steps vs. shortest path                         +0          +0
Distance (km)                                16.37       16.37
Optimality (% = optimal / found)            100.0%      100.0%
A* nodes expanded                                -           9
------------------------------------------------------------------
Greedy: Home -> Chợ Hoa Phượng -> Chợ Đầu Mối -> Siêu thị Xanh -> Home
A*    : Home -> Chợ Hoa Phượng -> Chợ Đầu Mối -> Siêu thị Xanh -> Home
```

At a small scale with 5 locations and 8 ingredients, both algorithms completed the search process with very low execution times. Greedy had a faster execution time than A*, at 0.000013 seconds and 0.000130 seconds, respectively. However, both algorithms found the same optimal route with a total distance of 16.37 km and 4 movement steps. Even though A* used more memory than Greedy, the results show both algorithms operate efficiently when the number of locations and ingredients is small.

b. 7 locations, 12 ingredients:

```
=== input2: 7 places, 12 ingredients (time = average of 20 runs) ===
Criterion                                   Greedy          A*
------------------------------------------------------------------
Average execution time (s)                0.000027    0.001024
Peak memory usage (bytes)                    2,216      51,888
Steps (moves, incl. return home)                 7           6
Steps vs. shortest path                         +1          +0
Distance (km)                                22.29       20.92
Optimality (% = optimal / found)             93.8%      100.0%
A* nodes expanded                                -          25
------------------------------------------------------------------
Greedy: Home -> Chợ Sớm -> Chợ Nhỏ Góc Phố -> Chợ Hải Sản -> Chợ Đông -> Chợ Trung Tâm -> Cửa hàng Tiện Lợi -> Home
A*    : Home -> Chợ Sớm -> Cửa hàng Tiện Lợi -> Chợ Trung Tâm -> Chợ Đông -> Chợ Hải Sản -> Home
```

When scaled up to 7 locations and 12 ingredients, the differences between the two algorithms become clearer. Greedy executed faster and used less memory than A*. However, Greedy produced a 22.29 km route with 7 steps, while A* only needed 20.92 km over 6 steps. A* achieved 100% optimality, whereas Greedy only reached 93.8%. This result demonstrates that although Greedy processes faster, A* is capable of finding shorter routes as the number of locations increases.

c. 10 locations, 15 ingredients:

```
=== input3: 10 places, 15 ingredients (time = average of 20 runs) ===
Criterion                                 Greedy          A*
------------------------------------------------------------------
Average execution time (s)              0.000068    0.002890
Peak memory usage (bytes)                  2,120     116,992
Steps (moves, incl. return home)               7           6
Steps vs. shortest path                       +1          +0
Distance (km)                              26.09       23.86
Optimality (% = optimal / found)           91.4%      100.0%
A* nodes expanded                              -          30
------------------------------------------------------------------
Greedy: Home -> Cửa hàng Tạp Hóa -> Chợ Sáng -> Chợ Thịt Phố -> Siêu thị Mini -> Chợ Vườn -> Chợ Cá Bến -> Home
A*    : Home -> Siêu thị Mini -> Chợ Thịt Phố -> Chợ Vườn -> Chợ Cá Bến -> Cửa hàng Tạp Hóa -> Home
```

With 10 locations and 15 ingredients, the gap between the outputs of the two algorithms continues to clearly show. Greedy found a 26.09 km route with 7 steps, whereas A* only needed 23.86 km with 6 steps. Greedy's optimality dropped to 91.4%, while A* maintained 100%. However, A* required considerably more processing time and memory than Greedy. Specifically, A*'s execution time was 0.002890 seconds, and memory used reached 116,992 bytes. This indicates A* yields more optimal outcomes but trades off against a larger computational footprint.

d. 12 locations, 18 ingredients:

```
=== input4: 12 places, 18 ingredients (time = average of 20 runs) ===
Criterion                                 Greedy          A*
------------------------------------------------------------------
Average execution time (s)              0.000037    0.003810
Peak memory usage (bytes)                  2,744     128,568
Steps (moves, incl. return home)               6           6
Steps vs. shortest path                       +0          +0
Distance (km)                              17.63       17.63
Optimality (% = optimal / found)           100.0%      100.0%
A* nodes expanded                              -          26
------------------------------------------------------------------
Greedy: Home -> Siêu thị Tiện Ích -> Chợ Hải Sản -> Chợ Thịt Phố -> Chợ Rau Xanh -> Siêu thị Sữa & Trứng -> Home
A*    : Home -> Siêu thị Tiện Ích -> Chợ Hải Sản -> Chợ Thịt Phố -> Chợ Rau Xanh -> Siêu thị Sữa & Trứng -> Home
```

At the scale of 12 locations and 18 ingredients, both algorithms discovered the exact same optimal route. The total distance for both was 17.63 km with 6 steps and 100% optimality. Nonetheless, Greedy retained its processing speed and memory footprint advantages. While Greedy only took 0.000037 seconds, A* took 0.003810 seconds and utilized 128,568 bytes of memory. This result shows that in certain cases, Greedy can still attain outcomes equivalent to A* without consuming substantial computational resources.

e. 15 locations, 24 ingredients:

```
=== input5: 15 places, 24 ingredients (time = average of 20 runs) ===
Criterion                                 Greedy          A*
------------------------------------------------------------------
Average execution time (s)              0.000080    0.007776
Peak memory usage (bytes)                  3,816     202,984
Steps (moves, incl. return home)              10           6
Steps vs. shortest path                       +4          +0
Distance (km)                              55.75       20.46
Optimality (% = optimal / found)           36.7%      100.0%
A* nodes expanded                              -          25
------------------------------------------------------------------
Greedy: Home -> Chợ Cầu -> Chợ Cá Bến -> Chợ Thịt Phố -> Siêu thị An Bình -> Siêu thị Mega -> Chợ Đồi -> Cửa hàng 24h -> Chợ Làng -> Tạp Hóa Cô Lan -> Home
A*    : Home -> Chợ Cầu -> Chợ Cá Bến -> Chợ Thịt Phố -> Siêu thị Hoàng Gia -> Chợ Đậu & Nấm -> Home
```

In the largest dataset with 15 locations and 24 ingredients, the discrepancy in route quality between the two algorithms is the most pronounced. Greedy generated a 55.75 km route taking 10 steps, whereas A* only needed 20.46 km with 6 steps. Greedy's optimality plummeted to just 36.7%, while A* achieved 100%. Even though Greedy had a much faster execution time and smaller memory footprint, the path it plotted was significantly longer. Conversely, A* found a shorter path with fewer steps but at the cost of higher time and memory usage. This clearly highlights the advantage of A* in finding optimal paths as the problem scale expands.

4.3. Performance Analysis

The comparative results indicate that each algorithm possesses its own strengths and is suited for different requirements in the shopping route problem:

Greedy (Nearest Neighbor) vs. A*

Algorithm	Path Optimality	Runtime Efficiency	Memory Usage	Suitability
Greedy	Variable (36.7%–100%)	Fastest overall	Low (1,664–3,816 bytes)	Fast route finding
A*	Always optimal (100% in all tests)	Slower as input size increases	High (7,808–202,984 bytes)	Finding the shortest route

In practical applications, the choice of algorithm depends on the goals and constraints of the problem. For multi-market grocery routing:

- **Greedy (Nearest Neighbor)** is suitable when there is a need for fast route generation while conserving computational resources, especially in scenarios where an absolutely minimal travel distance is not strictly required.
- **A*** is suitable when the absolute shortest route is demanded, such as planning multi-stop shopping trips to save time and commuting costs.

Overall, Greedy is the appropriate choice when processing speed is prioritized, while A* is better suited when route optimality is the main priority. The choice should be based on data scale and the specific goals of the problem at hand.

5. Discussion

5.1. Overall Trade-offs Evaluation

Summary of Experimental Data:

Input	Locations / Ingredients	Greedy Time (s)	A* Time (s)	A* Slower By	Greedy Memory (bytes)	A* Memory (bytes)	A* Memory Overhead	Greedy Distance (km)	A* Distance (km)	Greedy Optimality
1	5/8	0.000013	0.000130	~10 times	1,664	7,808	~4.7 times	16.37	16.37	100%
2	7/12	0.000027	0.001024	~38 times	2,216	51,888	~23 times	22.29	20.92	93.8%
3	10/15	0.000068	0.002890	~42 times	2,120	116,992	~55 times	26.09	23.86	91.4%
4	12/18	0.000037	0.003810	~103 times	2,744	128,568	~47 times	17.63	17.63	100%
5	15/24	0.000080	0.007776	~97 times	3,816	202,984	~53 times	55.75	20.46	36.7%

Three Axes of Trade-offs:

- **Processing Speed:** Greedy runs ~10 to 100 times faster than A* across all datasets. At each step, Greedy merely scans the market list once to pick the nearest one that still stocks missing ingredients, putting its time complexity at roughly $O(n^2)$ in the worst case. A* has to maintain a priority queue and calculate the heuristic for every new state. Each heuristic calculation iterates over all missing ingredients and the markets that sell them. The state space (position, missing ingredients set) can scale up to $n \cdot 2^k$, meaning A* theoretically grows exponentially.
- **Memory:** Greedy only stores the current location, the set of missing ingredients, and the route; $O(n)$, meaning its memory stays practically constant (1.6 KB to 3.8 KB). A* must store the priority queue (open_heap) and a table (best_g) (as seen in the code snippet in section 3.2.2.) to remember the shortest path found to each state, resulting in memory increasing from 7.8 KB to nearly 203 KB.
- **Accuracy:** Greedy only "looks one step ahead," making it prone to getting pulled into a nearby market that ultimately elongates the overall path. The results reveal:
    - Input 1 and 4 (favorable distribution of ingredients): Greedy matches the optimal path.

- Input 2 and 3: Greedy was longer by 1.37 km (+6.6%) and 2.23 km (+9.3%).
- Input 5 (widespread map, multiple rare ingredients): Greedy was longer by 35.29 km, translating to 2.7 times the optimal distance. Greedy has no backtracking mechanism or global evaluation, meaning its quality fluctuates wildly (36.7% to 100%). A*, with its admissible heuristic, always guarantees optimality (100% across all 5 datasets).

Observations on Trends:

- A*'s computational cost scales with size, but the value it provides scales as well: Between Input 1 and 5, A* is 10 times slower and then 97 times slower, but the distance saved jumps from 0 km to 35.29 km.
- Greedy's accuracy doesn't monotonically depend on scale: In Input 4 (12 markets), Greedy was optimal, while in Input 5 (15 markets), it failed heavily. The defining factor is the data structure (which market sells rare ingredients, and whether it's near or far), not just the sheer number of markets.
- Neither algorithm wins across all three axes: Greedy wins in speed and memory; A* wins in accuracy. This represents a genuine trade-off scenario.

Comparing Theoretical vs. Empirical Results:

Aspect	**Theory**	**Empirical Results**	**Practical Implications**
A* Complexity	May grow exponentially	Only expanded 9 to 30 states, max time ~7.8 ms	A good heuristic prunes effectively at household scale
A* Optimality	Optimal if h is admissible	100% in 5/5 cases	Matches theory
Greedy Optimality	Not guaranteed	36.7% - 100%	Matches theory, high risk with complex data
Memory	Greedy $O(n)$, A* grows with the number of states	1.6 KB - 3.8 KB compared to 7.8 KB - 203 KB	Matches theory, but 203 KB is still negligible for modern devices

Crucial Point:

At the scale of this research, the discrepancy in time and memory between the two algorithms is minuscule. At its slowest, A* still only takes about 0.008 seconds and roughly 203 KB of memory, completely imperceptible to human users. Conversely, **the difference in travel distance is massive** and **directly impacts gasoline usage and**

travel time. For instance, in dataset 5, Greedy's route is 55.75 km long, whereas A* requires only 20.46 km.

However, this outcome **only strictly applies to the small datasets our team tested**. We can observe that A*'s time and memory escalate relatively rapidly as the number of markets and ingredients goes up (time from 0.000130 s to 0.007776 s, memory from 7,808 bytes to 202,984 bytes). If the scale is significantly larger, A* might become resource-heavy, while Greedy would still execute fast because it only examines one step per iteration. This is a projection based on **the trend of the results**; our team has **not empirically verified this on massively scaled datasets**.

5.2. Real-world Applications and Implementation Scenarios

The insights gleaned from this multi-stop market routing problem naturally lend themselves to designing "Online Grocery" platform models that connect homemakers and delivery drivers (shippers). In practice, the core routing engine would run exclusively on the A* algorithm to ensure optimality, structured as follows:

- **Route Optimization:** The central system natively utilizes A* to calculate the global route from the onset, mapping out the absolute shortest path for the driver to fetch all requested items. Thanks to the heuristic estimation function, A* finds a globally optimal path rather than just reacting to what's immediately ahead.
- **Real-time Exception Handling:** The Greedy algorithm functions as a localized backup mechanism triggered by a singular exception: when the driver arrives at a store but the required item is unexpectedly out of stock. At that moment, to avoid having to re-compute a complex route from scratch, the mobile application uses Greedy to quickly reroute to the nearest available store carrying that exact item.
- **Operational Policy Simulation:** Allows platforms to simulate order allocation and accurately forecast commute expenses before deployment, purely relying on the 100% precise route generated by A*.

5.3. Case Study: Smart Grocery Routing at Grab

To demonstrate real-world applicability, we established a GrabMart service simulation scenario using the Input 5 dataset—a complex order featuring 24 ingredients distributed across a network of 15 markets/supermarkets. Within the system, travel distances between stops are calculated using 2-dimensional coordinates (Euclidean distance).

Simulation Design (Grab Driver Dispatch):

- The Grab Driver starts from their current position (Home, coordinates $(0, 0)$) upon accepting the order.
- The Grab backend servers deploy A* to issue a single, optimally shortest route spanning the required markets/supermarkets.
- At any designated stop, the driver acquires all remaining ingredients available at that specific location.

- **Exception handling:** If a store reports an out-of-stock item, the Grab Driver app instantly toggles to the Greedy algorithm, calculating the Euclidean distance from the driver's current coordinates to remaining stores to locate the nearest alternative vendor.

Insights and Implications:

The Input 5 scenario highlights the immense power of utilizing A* as the global dispatching system. Capable of mapping a globally optimal path, the Grab servers give the driver a baseline route of mere 20.46 km spanning 6 stops: Home → Chợ Cầu → Chợ Cá Bến → Chợ Thịt Phố → Siêu thị Hoàng Gia → Chợ Đậu & Nấm → Home (achieving a perfect 100% optimality). Had the system relied strictly on Greedy from the beginning, the driver would have been misled into a winding 55.75 km maze over 10 different locations.

However, Greedy's true value shines brightly as a localized exception handler. Assume the driver arrives at the very last stop on the route, **Chợ Đậu & Nấm (coordinates** $(4, 4)$**)**, to buy the final item, **Nấm**, only for the shopkeeper to declare they are out of stock. At this exact moment, instead of pinging the servers to re-run the intensive A* calculation (consuming 202,984 bytes of memory), the driver's mobile app instantly triggers Greedy to scan alternative locations stocking Nấm, which are Chợ Xanh (coordinates $(5, 3)$) and Siêu thị Mega (coordinates $(15, 2)$). The Greedy algorithm rapidly computes distances and reveals that Chợ Xanh is only about 1.41 km away, whereas Siêu thị Mega is a hefty 11.18 km away. The app instantly redirects the driver on a short detour to Chợ Xanh to buy the missing Nấm and head home.

With near-instantaneous processing speeds (only 0.000080 seconds), Greedy allows the app to react to out-of-stock events immediately without freezing the device or overloading the server. Through this architecture, the model beautifully marries theory and practice, fluidly blending algorithms to optimize delivery routing, handle operational risks, and slash travel costs for online grocery platforms.

6. Conclusion

6.1. Summary of the Study

a. Pros and Cons of each algorithm:

Greedy (Nearest Neighbor):

- **Pros:** Executes around 10 to 100 times faster than A*; its memory footprint is tiny and barely increases as scale goes up (from 1,664 to 3,816 bytes); simple to implement; the "choose the nearest market" logic closely mimics human intuition; and it can make instantaneous localized decisions if an anomaly like a stockout occurs.

- **Cons:** Fails to guarantee the shortest route; performance quality fluctuates wildly (between 36.7% and 100%); highly susceptible to being baited by nearby immediate stops, inadvertently elongating the macro-route.

A*:

- **Pros:** Always yielded the shortest path across all 5 datasets because the team's heuristic is admissible (it never overestimates the true cost).
- **Cons:** Consumes more time and memory than Greedy, with both metrics rising steadily alongside market and ingredient counts (time went from 0.000130 s to 0.007776 s; memory from 7,808 up to 202,984 bytes). The implementation logic is also notably more complex.

b. Conclusion on the Practicality of Greedy (Nearest Neighbor):

Assessing real-world viability, the team posits that **Greedy (Nearest Neighbor) is the most practical choice** for **scenarios requiring immediate**, **real-time responses**, whereas **A*** is distinctly **better for generating the overarching master route**.

First, **Greedy** is blindingly **fast** and **incredibly memory-light**. Across 5 datasets, Greedy maxed out at 0.000080 seconds and 3,816 bytes for computing the entire path, while A* ballooned to 0.007776 seconds and 202,984 bytes on the biggest dataset. Therefore, utilizing Greedy simply to ping the single nearest available market during a sudden anomaly is even lighter.

Second, **Greedy seamlessly fits into mid-trip disruptions**. When confronted with an out-of-stock item, the user immediately needs to know the closest alternative carrying it, rather than fully recalculating the entire itinerary. The logic of "closest alternative" is intuitive and easily deployable (see Section 5).

Third, **the empirical results confirm that Greedy performs reasonably well on small and medium datasets:** 100% optimal on Input 1 and 4, and 93.8% and 91.4% on Input 2 and 3, respectively.

Nevertheless, Greedy's flaws are glaring. On Input 5, Greedy's optimality cratered to 36.7%, tracing a 55.75 km path compared to A*'s 20.46 km. This happens because **Greedy only evaluates the immediate step without considering the subsequent path**, often causing it to drift far off-course.

Therefore, **A* is ideally suited for front-end global route generation as it guarantees the shortest possible distance**, while **Greedy acts as the perfect lightweight companion for handling mid-trip disruptions**. The two algorithms complement rather than replace one another, and our team advocates for a hybridized approach in real-world applications (see Section 5). Regarding future development, the team could run empirical tests simulating mid-route stockout scenarios to map out the extra distance Greedy tacks on versus a full A* recalculation, alongside substituting Euclidean distances with real-world road network metrics.

6.2. Future Works

Even though this study has established solid foundational grounds, there remains vast space for continued exploration and refinement in the future:

- **Hybrid Search Methods:** Dive deeper into melding the global integrity and guaranteed optimality of A* with the lightning-fast responsiveness of Greedy,

aiming to forge a convergent algorithm boasting both high accuracy and computational frugality.

- **Multi-objective Dynamic Heuristics:** Upgrade the estimation function by feeding it real-time variables—such as traffic congestion, supply chain disruption risks, and notably, price fluctuations. This allows the system to balance the absolute shortest travel distance against selecting vendors with **the most reasonable pricing**, thereby optimizing total costs for the end consumer.
- **Machine Learning for "Golden Time" Prediction:** Leverage Artificial Neural Networks to enable the system to autonomously parse historical order data, crowd behavior, and promotional periods. Consequently, the AI could accurately forecast the **"golden time" to shop** (e.g., supermarket flash sales, fresh food clearance hours, or off-peak lulls), driving dispatching decisions that are both incredibly cheap and aggressively minimize checkout wait times.
- **Real-World Implementation:** Directly embed these algorithmic engines into supply chain management systems or Internet of Things (IoT) networks within logistics platforms to conduct live field tests, benchmarking, and real-world validation.

6.3. Final Remarks

Ultimately, this study serves as vivid proof that classic AI search algorithms can absolutely be modernized and fiercely deployed into pressing, contemporary operational challenges, prominently including grocery routing and dispatch delivery. Within the scope of tested methodologies, the intersection of A*'s macro-vision and Greedy's flexible adaptability has risen as a superior, highly practical solution that establishes perfect harmony between network processing speeds and route quality.

The empirical testing on complex datasets (Input 5) further cements this declaration: heuristic-driven routing strategies do more than drastically slash commute times and aggressively optimize operational overhead for drivers. They also unlock enormous potential to safeguard the economic interests of consumers via mechanisms that prioritize vendors offering the best available pricing.

By dissolving the boundary between highly theoretical AI models and raw, real-world operational systems, this research contributes a critical puzzle piece to the sweeping panorama of smart logistics. In this arena, artificial intelligence doesn't stop at merely drawing lines on a map or boosting dispatch efficiency; it directly liberates manual labor, acting as an intelligent financial aide capable of hunting down the "golden times", thereby delivering tangible and enduring value to millions of homemakers and delivery drivers in their everyday lives.